28. a. Singly Linked List – TraverseListReverseOrderRecursive –Program Analysis and Time Complexity .

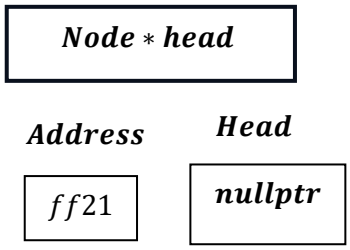
Program: –

```
void TraverseListReverseOrderRecursive(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {

        if ((*head)->next != nullptr)
        {
            TraverseListReverseOrderRecursive(&((*head)->next));
        }
        cout << (*head)->data << " ";
    }
}
….
 case 26:
        {
            cout << endl;
            cout << endl;
            cout << "Traverse List Reverse Order Recursive" << endl;
            TraverseListReverseOrderRecursive(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

1)*Linked List is Empty*:

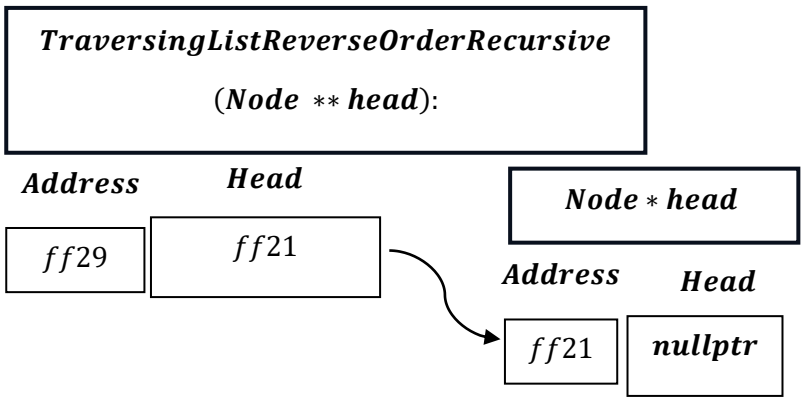
Pictorially depicted as:




```
if (*head == nullptr)
{
    cout << "List is empty." << endl;
    return;
}
```

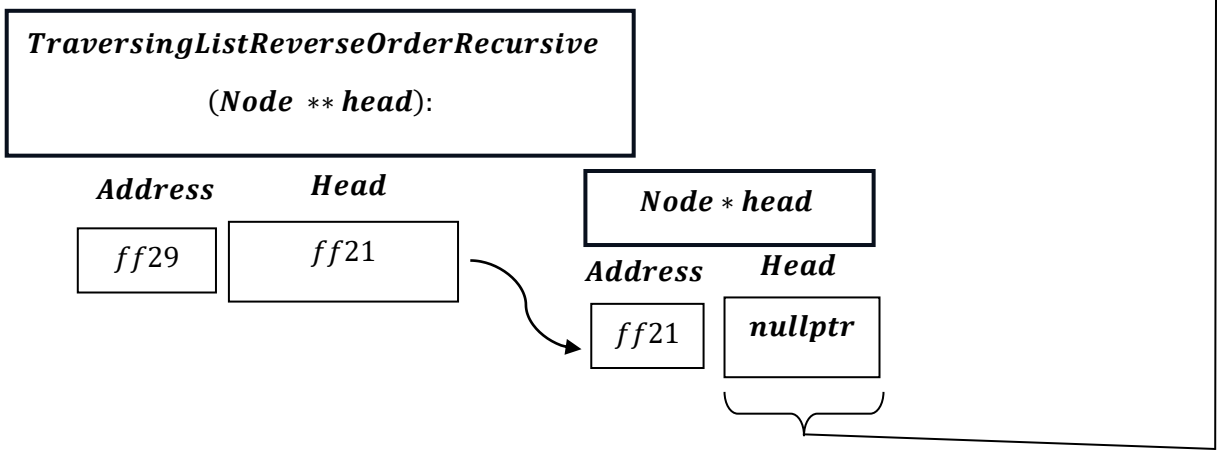
TraversingListReverseOrderRecursive(&***head***: ***ff21***);

```
void TraverseListReverseOrderRecursive (Node **head) {….}
```



if $(*$ ***head*** $==$ ***nullptr***$)$ ***if it is true***

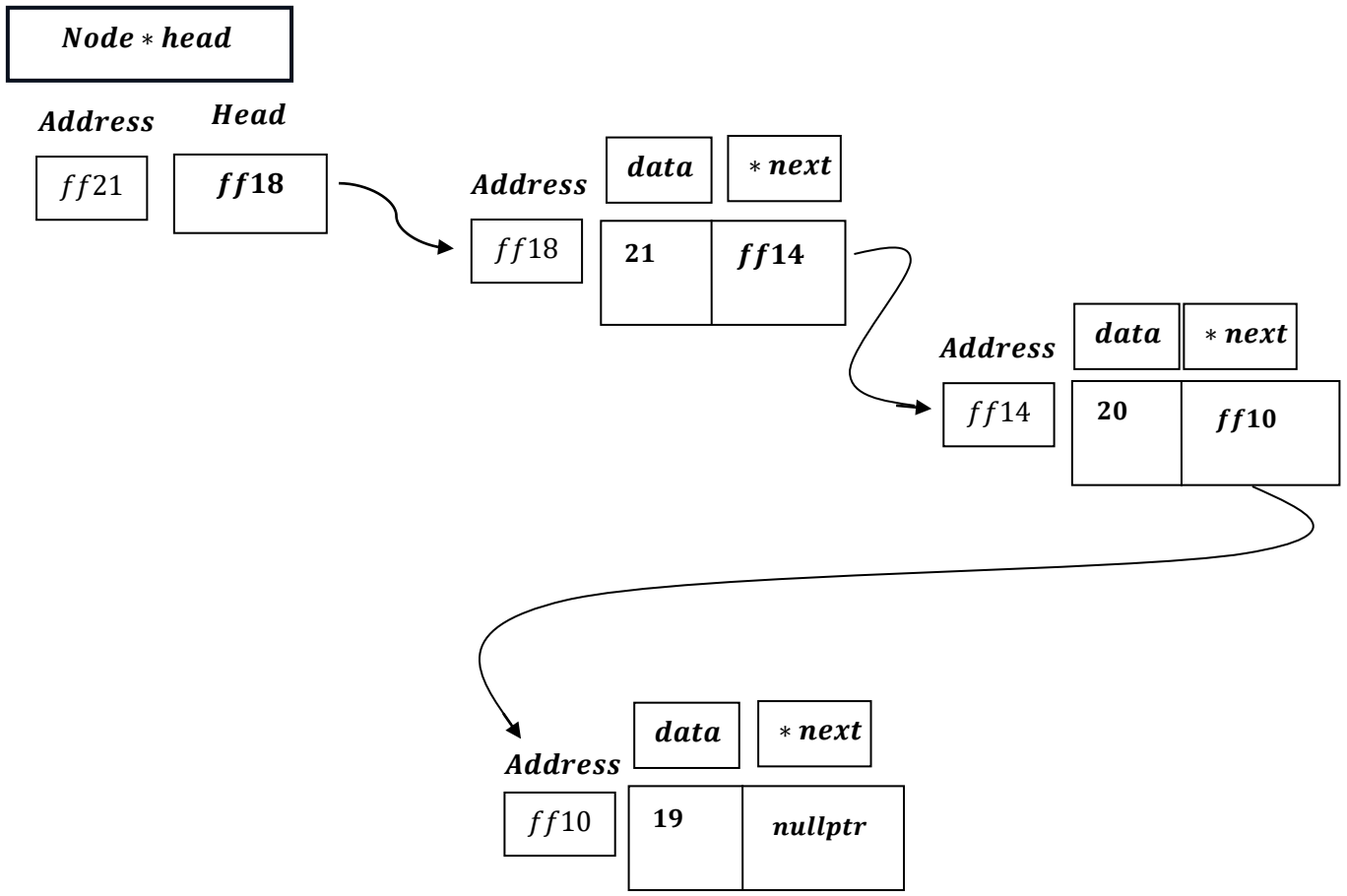
If true then Why true ?
Ans: ***Though*** $*$ ***head which is of Node*** $**$ ***head***
temporary pointer to pointer variable of the function
, as $*$ ***head is used , it dereference from Node*** $**$ ***head and***
refer to Node $*$ ***head global pointer to which Node*** $**$ ***head***
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the `TraversingListReverseOrderRecursive` function.



If List is Empty , then , it prints a message : `Error: List is empty.` and returns nothing as function has void data type . And function ends.

---

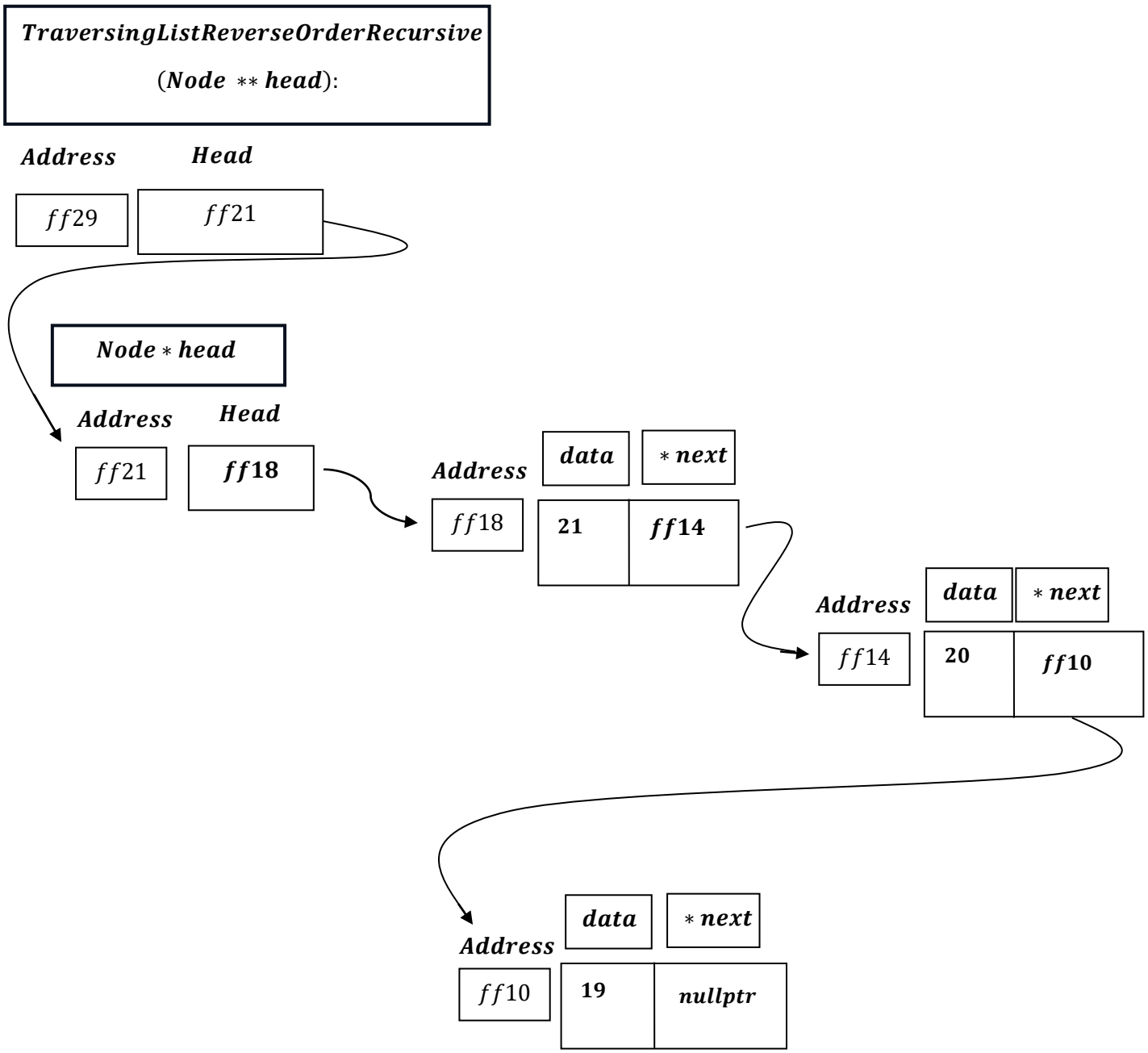
2)*Linked List is not Empty*:



Function Call:

```
TraverseListReverseOrderRecursive(&head);
```

i.e.TraversingListReverseOrderRecursive(&***head***:***ff21***)



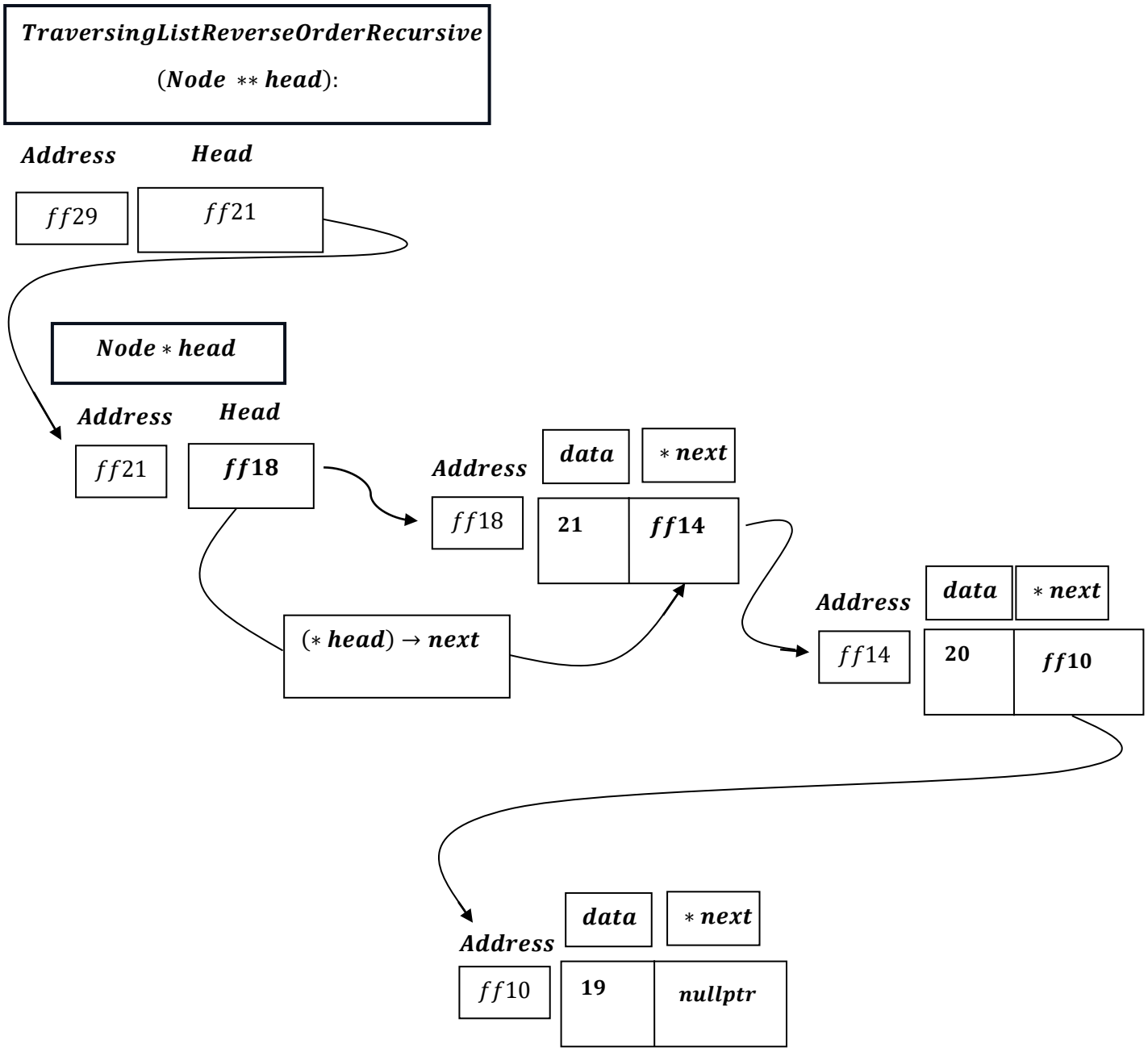

```
void TraverseListReverseOrderRecursive(Node **head) {
    if (*head == nullptr){ ….}

    else
    {
```

```
if ((*head)->next != nullptr)
        {
            TraverseListReverseOrderRecursive(&((*head)->next));
        }
        cout << (*head)->data << " ";
    }
}
```

```
if ((*head)->next != nullptr) {….}
```

i.e.,



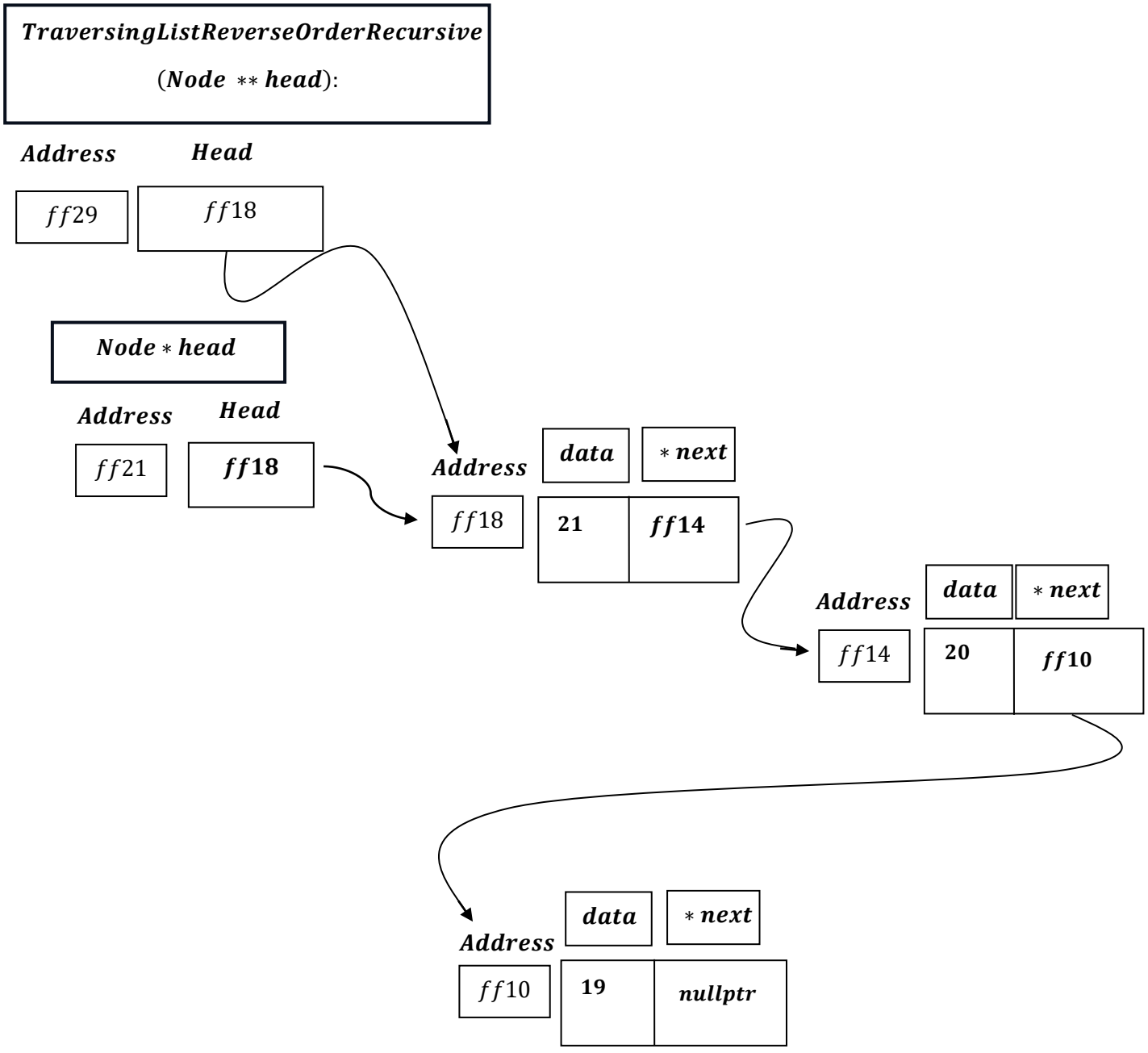
Node * head has value i.e. Address → ff18 and ff18 → next is ff14 i.e. not nullptr, then:

Recursive Function call:

```
TraverseListReverseOrderRecursive(&((*head)->next));
```

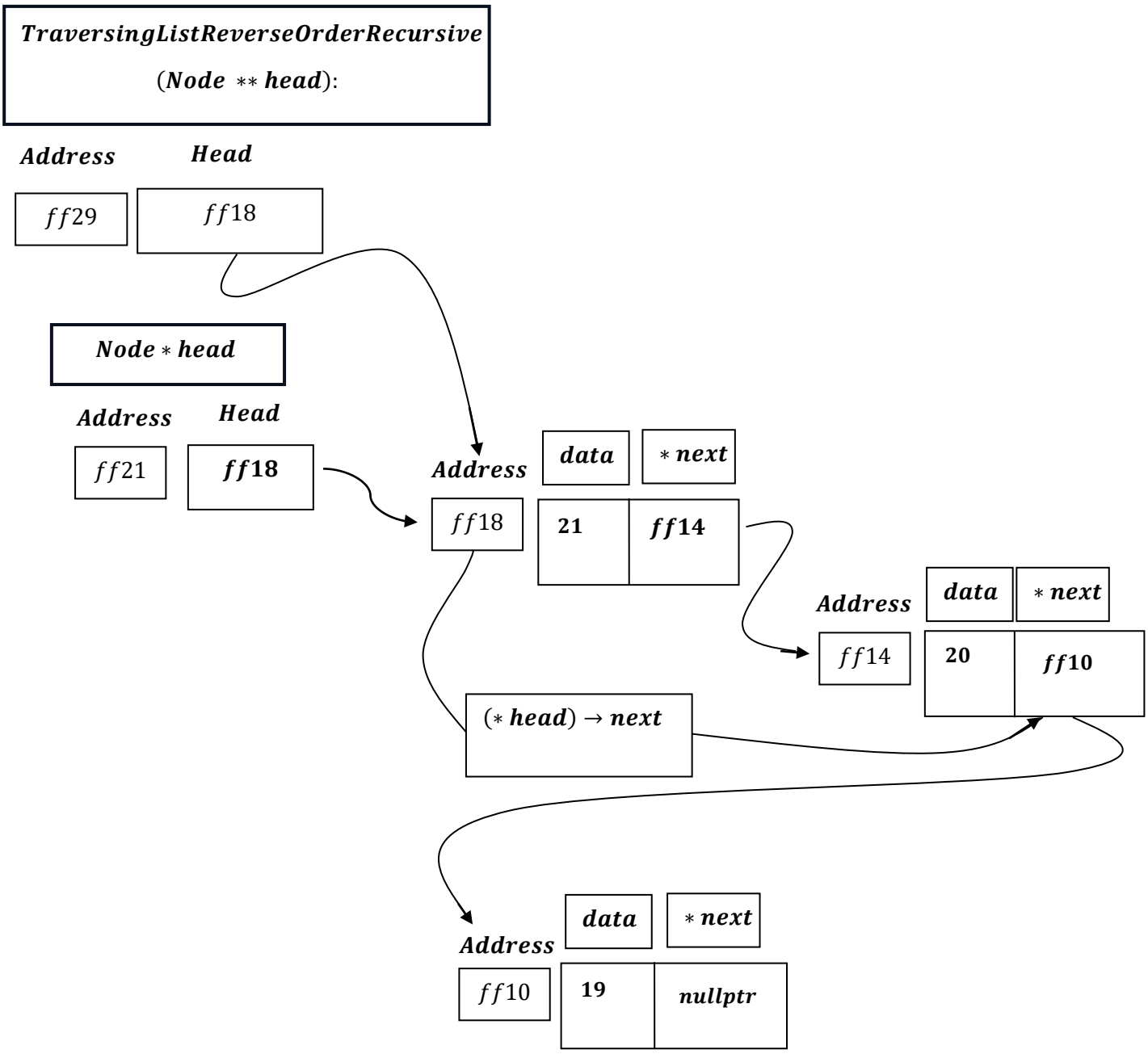
ff50	$Node ** head = \&((* head) \rightarrow next) = ff18$
ff46	***Main Func:*** $Node ** head = \&head = ff21$

i.e. in Stack it will store the address where $(* head) \rightarrow next;$ ***→ ff14 is stored and*** $Node ** head = \&((* head) \rightarrow next) = \&(ff14) = ff18.$



```
if ((*head)->next != nullptr) {….}
```

i.e.,



Note, now, Node ** Head doesnot point to Node * Head , therefore , now , (* head) will dereference from address ff18 done by dereference operator `` * `` and Node * will point to ff14 and (* head) → next; reflects ff14 → next i.e., ff10 which is not nullptr , hence again recursive call takes place.

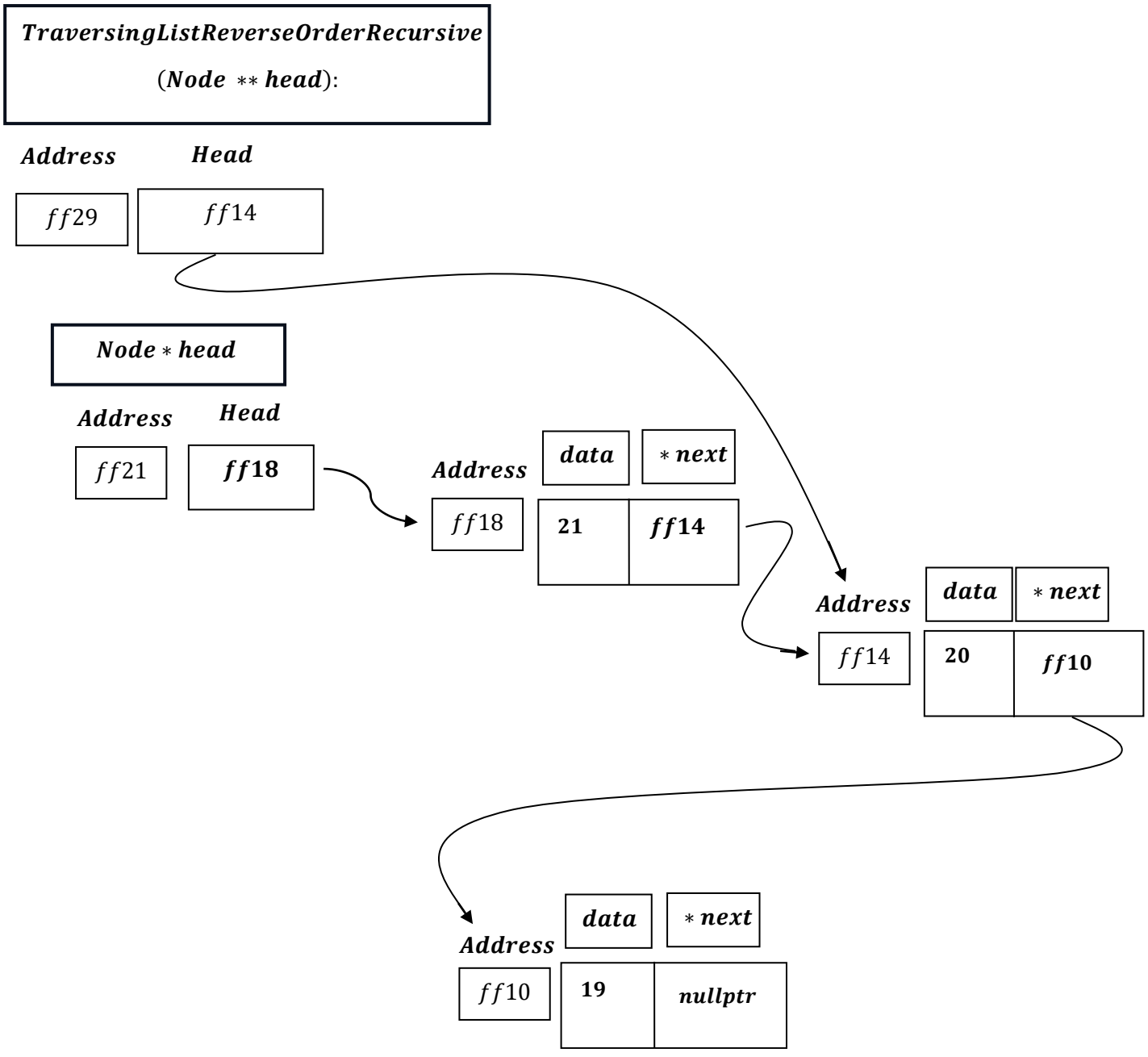
Recursive Function call:

```
TraverseListReverseOrderRecursive(&((*head)->next));
```

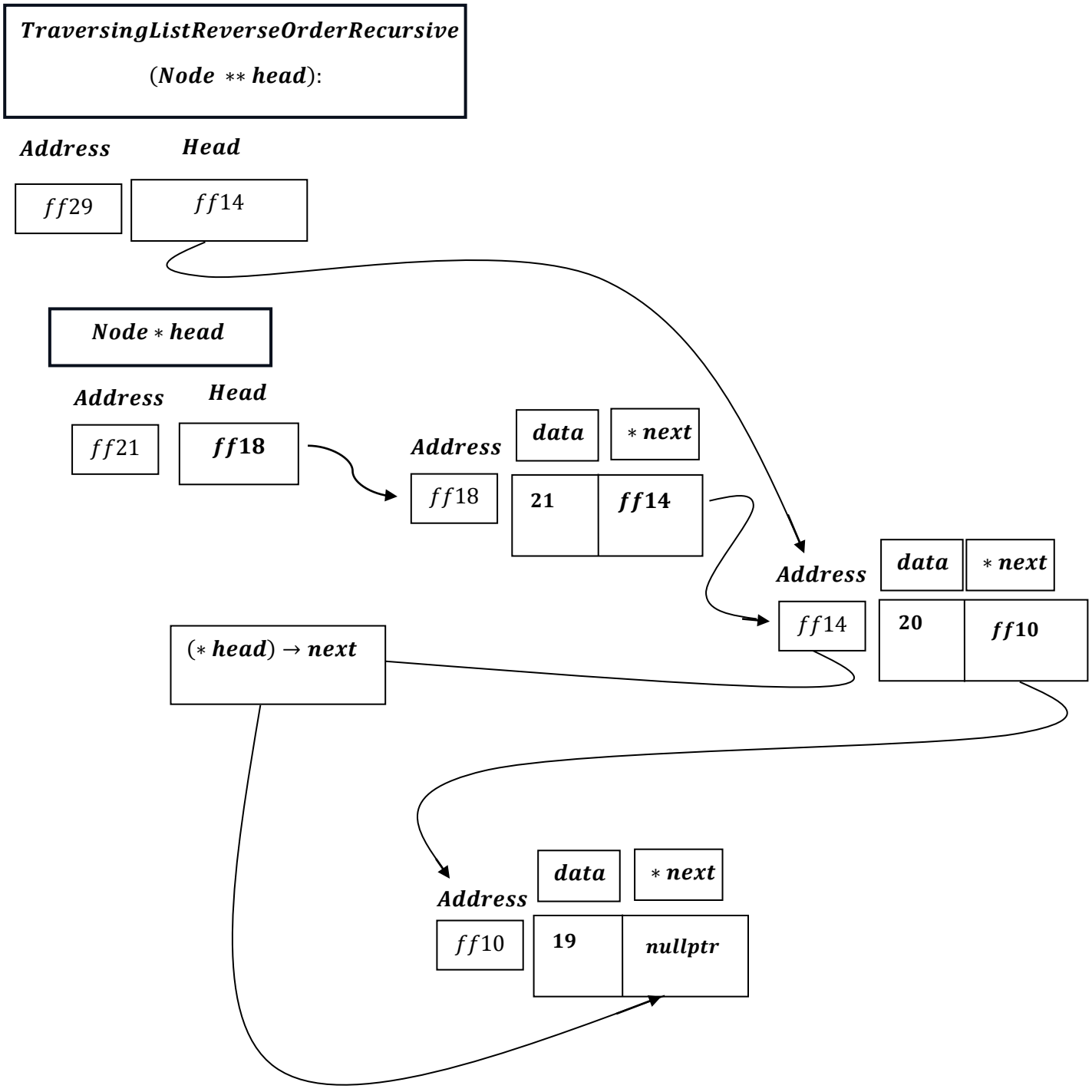
$ff54$	$Node ** head = \&((* head) \rightarrow next) = ff14$
$ff50$	$Node ** head = \&((* head) \rightarrow next) = ff18$
$ff46$	$Main\ Func: Node ** head = \&head = ff21$

$i.e.\ in\ Stack\ it\ will\ store\ the\ address\ where\ (* head) \rightarrow next; \rightarrow ff10\ is\ stored\ \ and$

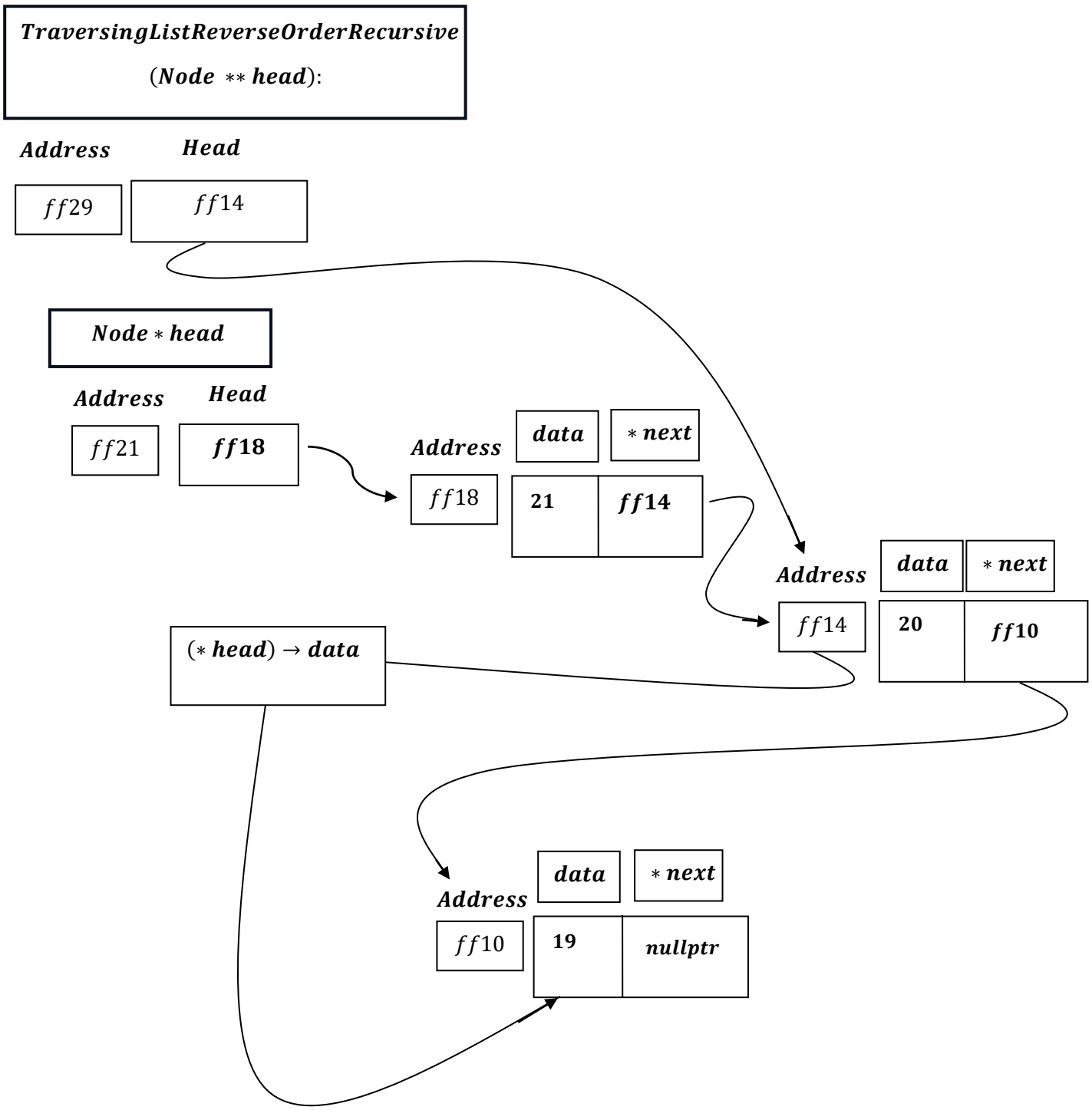
$Node ** head = \&((* head) \rightarrow next) = \&(ff10) = ff14.$



Note, now, Node ** Head doesnot point to Node * Head , therefore , now , (* head) will dereference from address ff14 done by dereference operator `` * `` and Node * will point to ff10 and (* head) → next; reflects ff14 → next i.e., ``nullptr`` which is nullptr , hence again recursive will stop here and exit.

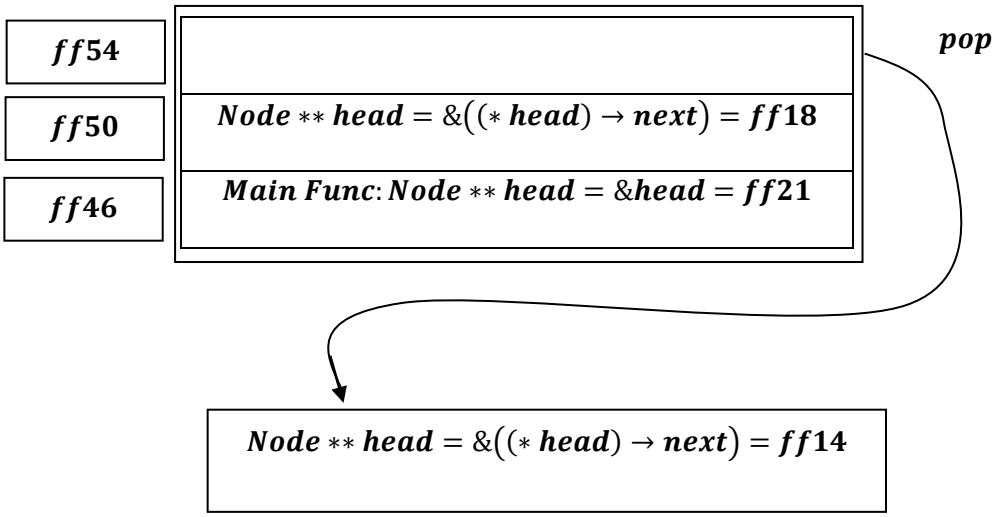


Now, we will print $(*\ \boldsymbol{head}) \rightarrow \boldsymbol{data}$;



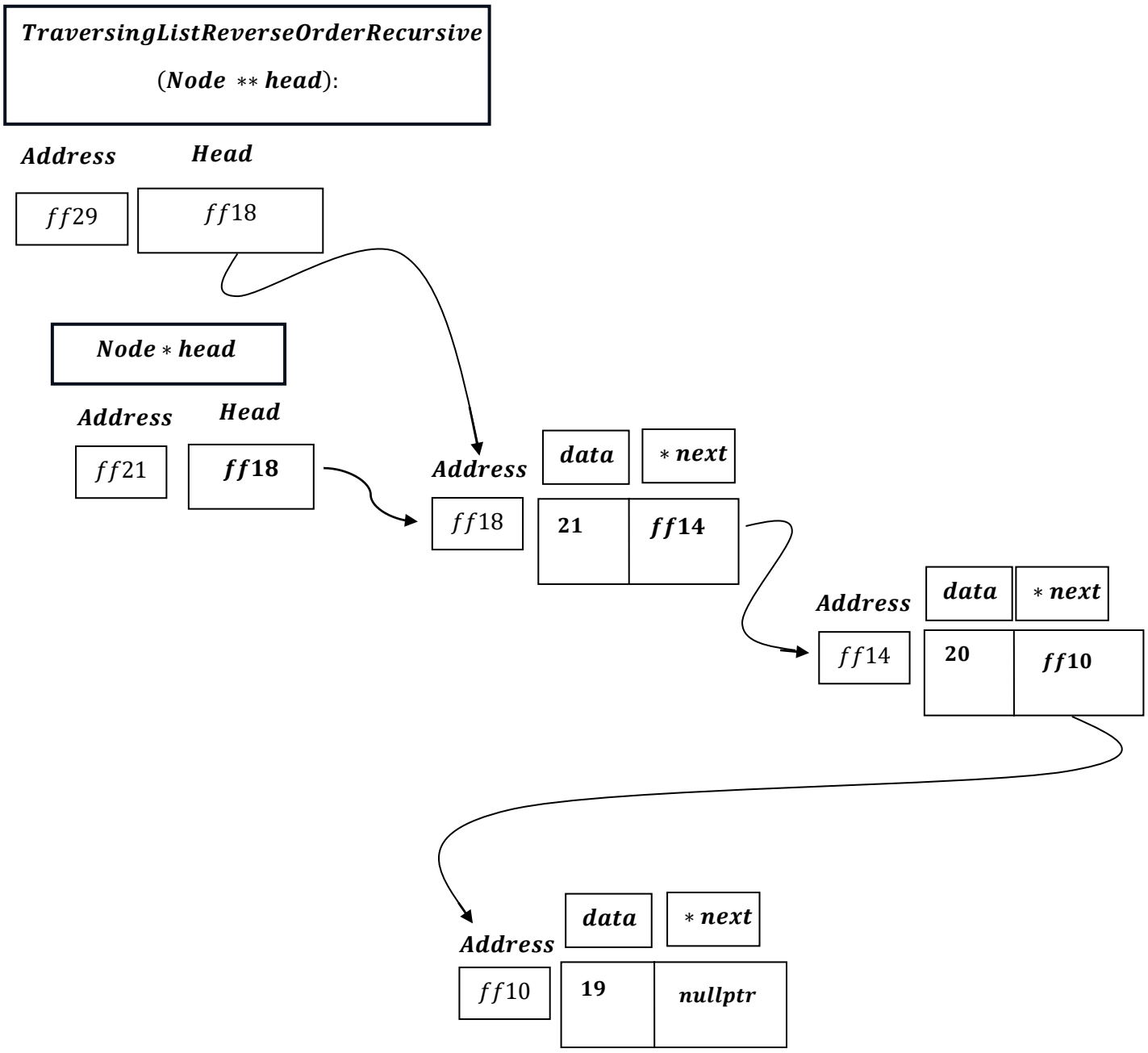
Hence $(*\ \boldsymbol{head}) \rightarrow \boldsymbol{data}$; ***will print*** **19**.

$Next, Node ** head = \&((* head) \rightarrow next) = ff14, will\ pop\ out\ from\ the\ stack$.

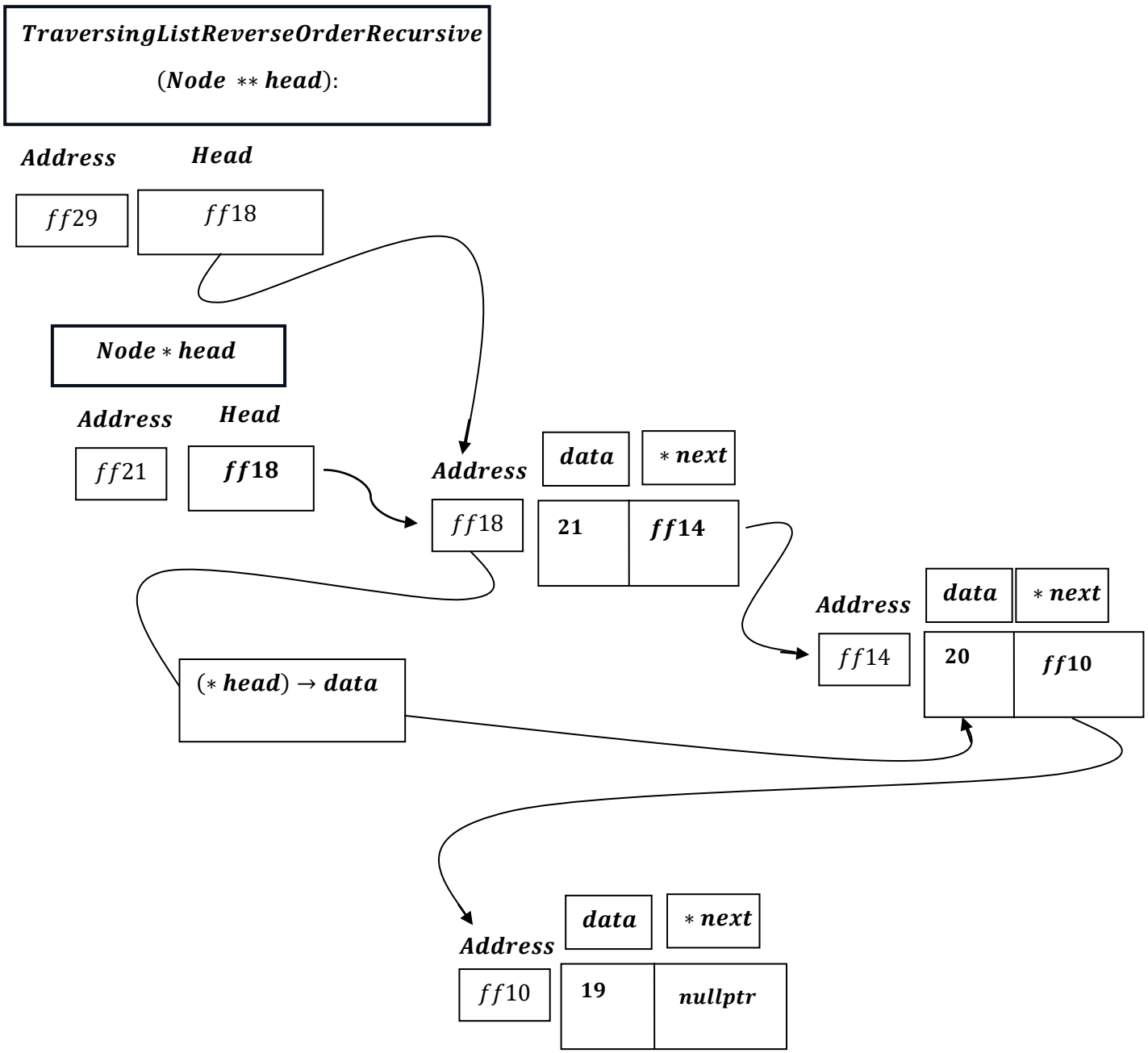


$Now, in\ the\ stack\ it\ remains, Node ** head = ff18, hence\ Node ** head\ will\ get\ assigned\ to\ ff18.$

$i.e.$

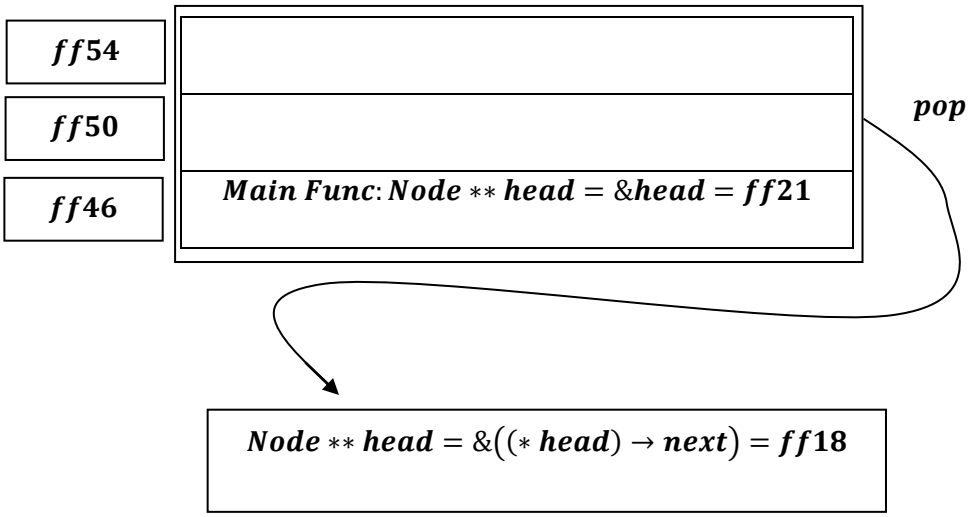

TraversingListReverseOrderRecursive
(Node ** head):
Address
Head
ff29
ff18
Node * head
Address
Head
ff21
ff18
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10
Address
data
* next
ff10
19
nullptr

Now, we will print $(*\ \boldsymbol{head}) \rightarrow \boldsymbol{data}$;



Hence $(*\ \boldsymbol{head}) \rightarrow \boldsymbol{data}$; ***will print*** **20**.

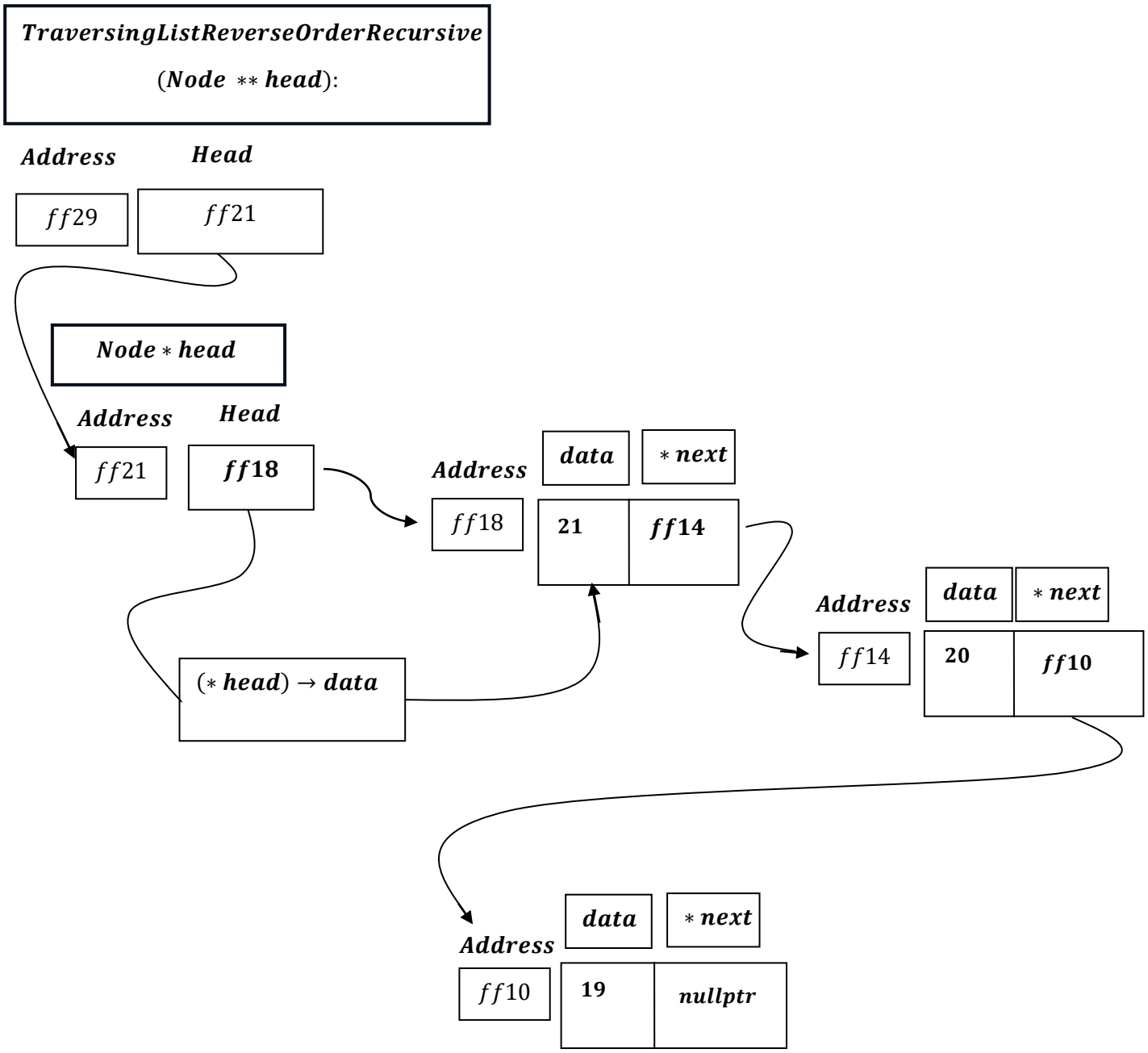
$Next, Node ** head = \&((* head) \rightarrow next) = ff14, will\ pop\ out\ from\ the\ stack$.



$Now, in\ the\ stack\ it\ remains, Node ** head = ff21, hence\ Node ** head\ will\ get\ assigned\ to\ ff21.$

$i.e.$

Now, we will print $(*\ \boldsymbol{head}) \rightarrow \boldsymbol{data}$;



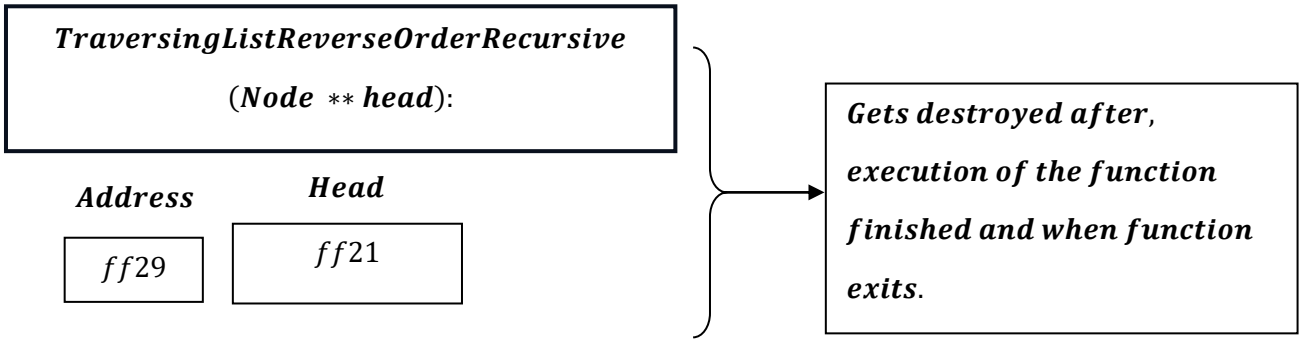
As $\boldsymbol{Node} ** \boldsymbol{head} = \&\boldsymbol{head}$ ***i.e address of*** $\boldsymbol{Node} * \boldsymbol{head} = \boldsymbol{ff21}$, ***now hence,***

$(* \boldsymbol{head}) \rightarrow \boldsymbol{data}$, ***here,*** $* \boldsymbol{head}$ ***reflects*** $\boldsymbol{Node} * \boldsymbol{head}$ ***i.e.*** $\boldsymbol{ff18}$ ***and*** $(* \boldsymbol{head}) \rightarrow \boldsymbol{data}$ ***is***:

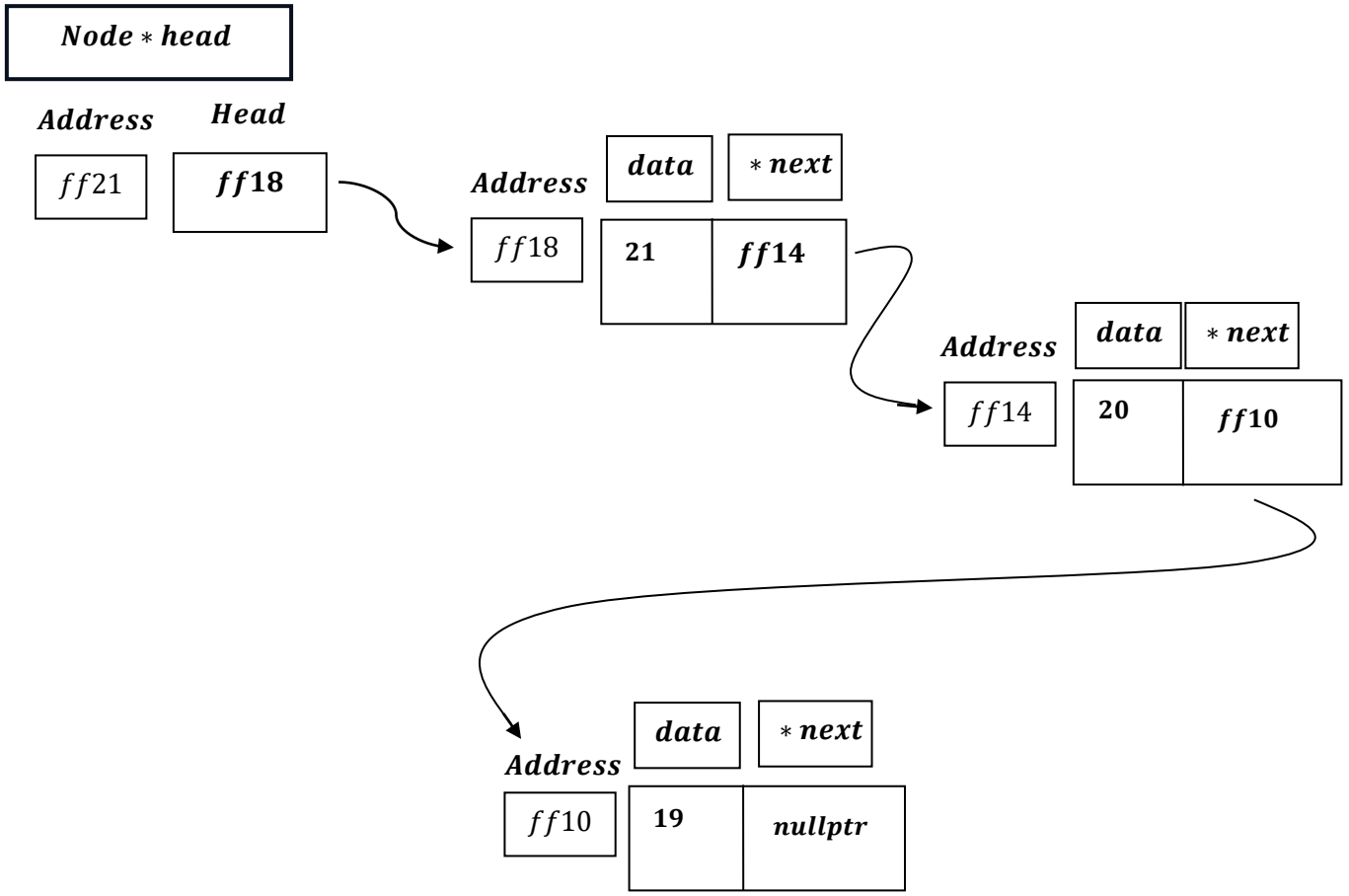
$\boldsymbol{ff18} \rightarrow \boldsymbol{data}$.

Hence $(* \boldsymbol{head}) \rightarrow \boldsymbol{data}$; ***will print*** $\boldsymbol{21}$.

Now, as the function get finished, all temporary variable created earlier in process for the program will get destroyed.



And we are left with:



The function TraversingListReverseOrderRecursive step by step analysis:

1. ***Function Signature:*** **–**

- ***Parameter:*** ***Node ** head – a pointer to a pointer to the head of the list. This double pointer is used to maintain a reference to the original head node of the list throughout the recursive calls.***

2. ***Base Case Check for Empty List:*** **–**

- ***This checks if the list is empty by verifying if the head pointer points to nullptr.***
- ***If the list is empty, it prints "List is empty." and exits the function.***

3. ***Recursive Case:***

- ***Condition:*** $if\ ((*head) \rightarrow next\ != nullptr)$:
  - ***This checks if the current node*** $(*head)$***is not the last node in the list. If it's not the last node, the function makes a recursive call.***

- ***Recursive Call:*** $TraverseListReverseOrderRecursive\left(\&((*head) \rightarrow next)\right)$:
  - ***The function is called recursively with the address of the next pointer of the current node. This moves the recursion one step further along the list.***
  - ***This recursive call continues until the end of the list is reached*** $(i.e., when\ (*head) \rightarrow next\ is\ nullptr)$.
- ***Print Statement:*** $cout\ \ll\ (*head) \rightarrow data\ \ll\ ;$
  - ***After the recursive call returns*** $(which\ happens\ when\ the\ end\ of\ the\ list\ is\ reached\ and\ the\ recursion\ unwinds)$***, the data of the current node is printed.***
  - ***This print statement is executed after the recursive call, which means it prints the nodes in reverse order as the recursion unwinds back up the call stack.***

TraversingListReverseOrderRecursive

−Time Complexity .

Best Case (When List is Empty)

When List is Empty, then, as soon as it enters the function: ***TraversingListReverseOrderRecursive, if checks whether head pointer pointed to ``nullptr`` or not , if so then outputs: ``List Is Empty`` and returns nothing , and then exits from function. This takes, constant `c` amount of time i.e.*** $\Omega(1)$.

Hence when List is Empty, we get our best case : $\Omega(1)$.

$\underline{Worst\ Case\ (When\ List\ is\ not\ Empty)}$

$\underline{Base\ Case}$:

$When\ T(1)\ i.e\ When\ head\ will\ be\ pointing\ last\ node, then\ head \rightarrow next = nullptr$ $recursion\ exits\ which\ we\ \ say\ , base\ case, as\ pointing\ last\ node, recursion\ exit$ $hence\ T(1) = 1.$

$\underline{Recursion}$:

$The\ recursion\ occurs\ T(n-1)\ times\ i.e.if\ n = 3\ then\ 2\ times\ as\ third\ time$ $(*head) \rightarrow next;\ will\ be\ ``nullptr``.$

$And, on\ unwinding\ of\ Recursion, each\ time\ it\ prints\ (*head) \rightarrow data\ ;.$

$That\ is\ forming: T(n-1) + 1$

$Also, some\ may\ say, using\ Base\ Case: T(n-1) + T(0)\ , but\ more\ precise\ way\ is$ $using, printing\ \ (*head) \rightarrow data\ ; on\ unwinding\ of\ recursive\ stack\ .$

$$T(n) = \begin{cases} 1 & , for\ (n = 1) \\ T(n-1) + 1 & , for\ (n > 1) \end{cases}$$

$A)$**By Substitution Method**:

$T(n) = T(n-1) + 1$

Therefore , $T(n-1) = T(n-2) + 1$

Substituting this in $T(n)$ **we get**:

$T(n) = T(n-2) + 1 + 1 = T(n-2) + 2$

Now , $T(n-2) = T(n-3) + 1$

Substituting this in $T(n)$ **we get**:

$T(n) = T(n-3) + 1 + 2 = T(n-3) + 3$

Now if it runs upto `k` times we get$(k$ **levels**$)$:

Now , $T(n) = T(n-k) + k$

We know, base case $T(1) = 1$, **hence**, $n - k = 1 \implies k = n - 1$.

When $k = n$, **we get**:

Now , $T(n) = T(n-(n-1)) + (n-1)$

$= T(1) + n - 1$

$= 1 + n - 1$

$= n$

Applying , **Big `O`** , **we get**:

$= O(n)$.

B)Recursion Tree Method:

$$T(n) = \begin{cases} 1 & ,for\ (n = 1) \\ T(n-1) + 1 & ,for\ (n > 1) \end{cases}$$

or, we can say:

$$T(n) = \begin{cases} 1 & ,for\ (n = 1) \\ T(n-1) + C & ,for\ (n > 1) \end{cases}$$

Recalling program:

```
void TraverseListReverseOrderRecursive(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {

        if ((*head)->next != nullptr)
```

```
        {
            TraverseListReverseOrderRecursive(&((*head)->next));
        }
        cout << (*head)->data << " ";
    }
}
```

Recursive Call:

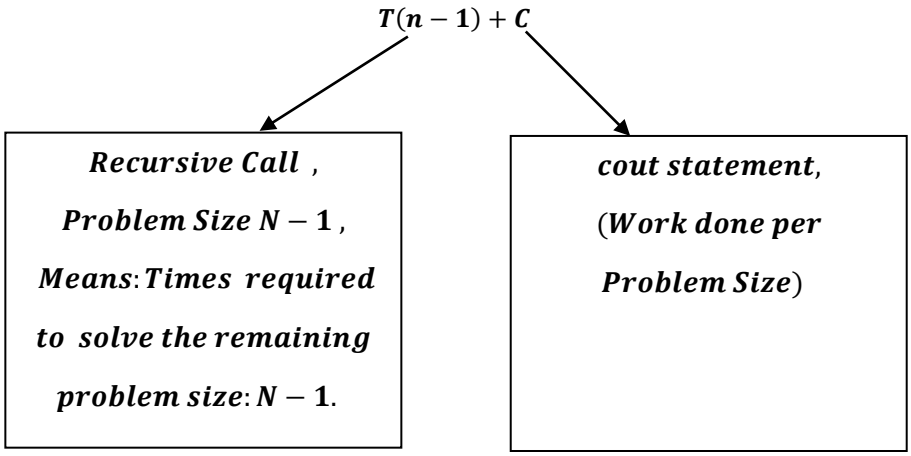
```
            TraverseListReverseOrderRecursive(&((*head)->next));
```

$runs\ T(N-1)\ times.$

Work done per Problem Size:

```
        cout << (*head)->data << " ";
```

$This\ print\ statement\ takes\ `C`\ , constant\ amount\ of\ time.$



$Level$	$No.of\ problems$	$Problem\ Size$	$Work\ /Cost\ Per\ Problem$ ($Print$ `cout` $statement$)	$Work\ done = Work\ per\ Problem \times No.of\ Problems$
0	1	n	C	$1 \times C = C$
1	1	$n-1$	C	$1 \times C = C$
2	1	$n-2$	C	$1 \times C = C$
. .	. .	. .	. .	. .
$n-1$ $(Base\ Case)$	1	$1(Base\ Case)$	$T(1)$	$1 \times T(1) = T(1)$

$Why\ level\ `n-1`\ is\ Base\ Case?$
$Ans: for, level\ `k`:, Base\ Case = T(1) \therefore n-k=1$
$\Rightarrow k = n-1.$

$Total\ Cost = C + C + C + \cdots (N-1) times + T(1).$
$= C(N-1) + T(1)$
$We\ know, T(1) is\ 1\ instead\ lets\ write\ T(1) = C_1,$
$representing\ constant.$

$= C(N-1) + C_1$

$As, C\ and\ C_1\ both\ are\ constant\ Applying\ Big\ O, we\ get:$

$O(N-1)$
$= O(N)$

$\underline{\textbf{Average Case}(\textbf{Average Case} = \textbf{Worst Case})}$

The average case for traversing a linked list typically involves traversing a list of average length. Since the function must visit each node to print its data, the time complexity is $(O(n))$, ***where*** (n) ***is the number of nodes in the list.***

$\textbf{Note , by relation: } \Omega \leq \Theta \leq O \textbf{ , we get:}$

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq n \leq c_2 \times n$

Singly Linked List	***Cases***	***Time Complexity***
$\textbf{TraversingListReverseOrderRecursive}$	**1.** $\textbf{Best Case}$	$\Omega(1)$
	2. $\textbf{Worst Case}$	$O(n)$
	3. $\textbf{Average Case} = \textbf{Worst Case}$	$\Theta(n)$

****